

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

**Отчёт
Лабораторная работа №2**

Выполнил:
Степанов Виктор
J3213

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Подключить ранее разработанное приложение (ЛР1, вариант 5 — AI Model & Dataset Hub) к внешнему API средствами `fetch`. Реализовать моковое API средствами **JSON-сервера** и подключить к нему авторизацию.

Теоретическая часть

Что такое API

API (Application Programming Interface) — интерфейс взаимодействия между клиентом (браузером) и сервером. Клиент отправляет HTTP-запросы, сервер возвращает данные в формате JSON.

HTTP-методы:

- GET — получить данные (список моделей, информацию о пользователе)
- POST — отправить данные (логин, регистрация)
- PUT — обновить запись
- DELETE — удалить запись

JSON — формат данных

JSON (JavaScript Object Notation) — текстовый формат обмена данными. Используется для передачи данных между клиентом и сервером:

```
{
  "id": 1,
  "name": "ru-bert-base",
  "task": "NLP",
  "stars": 1240
}
```

fetch — встроенный метод браузера

`fetch` — встроенный в браузер метод для выполнения HTTP-запросов. Возвращает Promise (асинхронный результат):

```
const response = await fetch('http://localhost:3001/models');
const data = await response.json();
```

Отличие от axios: `fetch` встроен в браузер, не требует установки. `axios` — сторонняя библиотека, удобнее для сложных случаев (interceptors, автопарсинг JSON). В ЛР2 используется `fetch`, в ЛР3 — `axios`.

Токен авторизации

При успешном входе сервер возвращает токен — уникальную строку, доказывающую что пользователь авторизован. Токен хранится в `localStorage` браузера и отправляется с каждым запросом в заголовке **Authorization**:

```
headers: {
  'Authorization': 'Bearer ' + localStorage.getItem('token')
}
```

Зачем localStorage? Данные в localStorage сохраняются между перезагрузками страницы. Если хранить токен в обычной переменной — при перезагрузке пользователь выйдет из системы.

json-server — моковый API

json-server — Node.js библиотека которая читает `db.json` и автоматически создаёт REST API. Используется для разработки фронтенда без написания настоящего бэкенда.

Запуск:

```
npm install json-server@0.17.4
node server.js
# JSON Server                                http://localhost:3001
```

Автоматические роуты из db.json:

- GET /models — вернуть все модели
- GET /models/1 — вернуть модель с id=1
- POST /models — создать новую модель

Дополнительные роуты (/login, /register, /me, /logout) реализованы вручную в `server.js`.

CORS

CORS (Cross-Origin Resource Sharing) — механизм безопасности браузера. По умолчанию браузер блокирует запросы к другому домену/порту. json-server разрешает CORS автоматически, поэтому запросы с `localhost:5500` к `localhost:3001` работают без ошибок.

Ход работы

Структура проекта ЛР2

- `db.json` — база данных: пользователи, модели (10 штук), токены
- `server.js` — сервер на основе json-server с кастомными роутами авторизации
- `public/index.html` — фронтенд (обновлённый index.html из ЛР1)
- `package.json` — зависимости Node.js проекта

Что изменилось по сравнению с ЛР1

1. Данные из сервера, не из массива. В ЛР1 модели захардкожены в JS-массиве. В ЛР2 при загрузке страницы делается GET-запрос к серверу:

```
async function loadModels() {
  try {
    MODELS = await apiFetch('/models');
  } catch(e) {
    //

    MODELS = [ ... ];
  }
}
```

2. Настоящая авторизация через сервер. В ЛР1 вход принимал любые данные. В ЛР2 сервер проверяет email и пароль в db.json:

```
// server.js
server.post('/login', (req, res) => {
  const { email, password } = req.body;
  const user = db.get('users')
    .find({ email, password }).value();
  if (!user) {
    return res.status(401)
      .json({ error: 'Неверный email или пароль' });
  }
  const token = 'token_' + user.id + '_' + Date.now();
  db.get('tokens').push({ token, userId: user.id }).write();
  res.json({ token, user: { ... } });
});
```

3. Единая функция apiFetch с токеном. Все запросы к API проходят через функцию apiFetch, которая автоматически добавляет токен:

```
async function apiFetch(url, options = {}) {
  const token = localStorage.getItem('token');
  const headers = { 'Content-Type': 'application/json' };
  if (token) headers['Authorization'] = 'Bearer ' + token;
  const res = await fetch(API + url, { ...options, headers });
  const data = await res.json();
  if (!res.ok) throw new Error(data.error || 'Ошибка сервера');
  return data;
}
```

4. Сессия сохраняется после перезагрузки. При загрузке страницы вызывается checkSession() — проверяет токен в localStorage через GET /me:

```
async function checkSession() {
  const token = localStorage.getItem('token');
  if (!token) return;
  try {
    const user = await apiFetch('/me');
    loggedIn = true;
    currentUser = user.username;
    updateNav();
  } catch(e) {
    //
  }
}
```

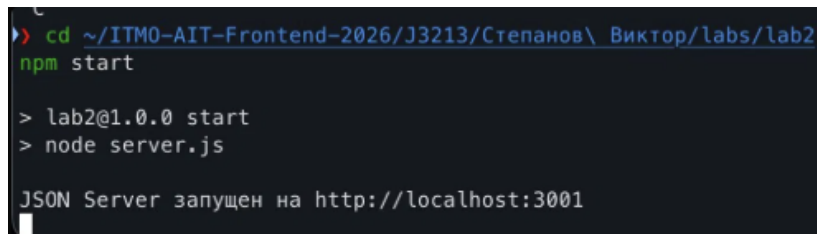
```
    localStorage.removeItem('token');  
  }  
}
```

Данные в db.json

В db.json хранятся 3 коллекции:

- **users** — пользователи (email, пароль, username). Тестовый пользователь: vtector@itmo.ru / password123
- **models** — 10 записей (6 моделей + 4 датасета). Данные фиктивные — придуманы для демонстрации
- **tokens** — активные токены сессий (создаются при входе, удаляются при выходе)

Результаты



```
> cd ~/ITMO-AIT-Frontend-2026/J3213/Степанов\ Виктор/labs/lab2  
npm start  
  
> lab2@1.0.0 start  
> node server.js  
  
JSON Server запущен на http://localhost:3001
```

Рис. 1: Запуск json-server командой `npm start`. Сервер поднимается на `localhost:3001` и отдаёт данные из `db.json`.

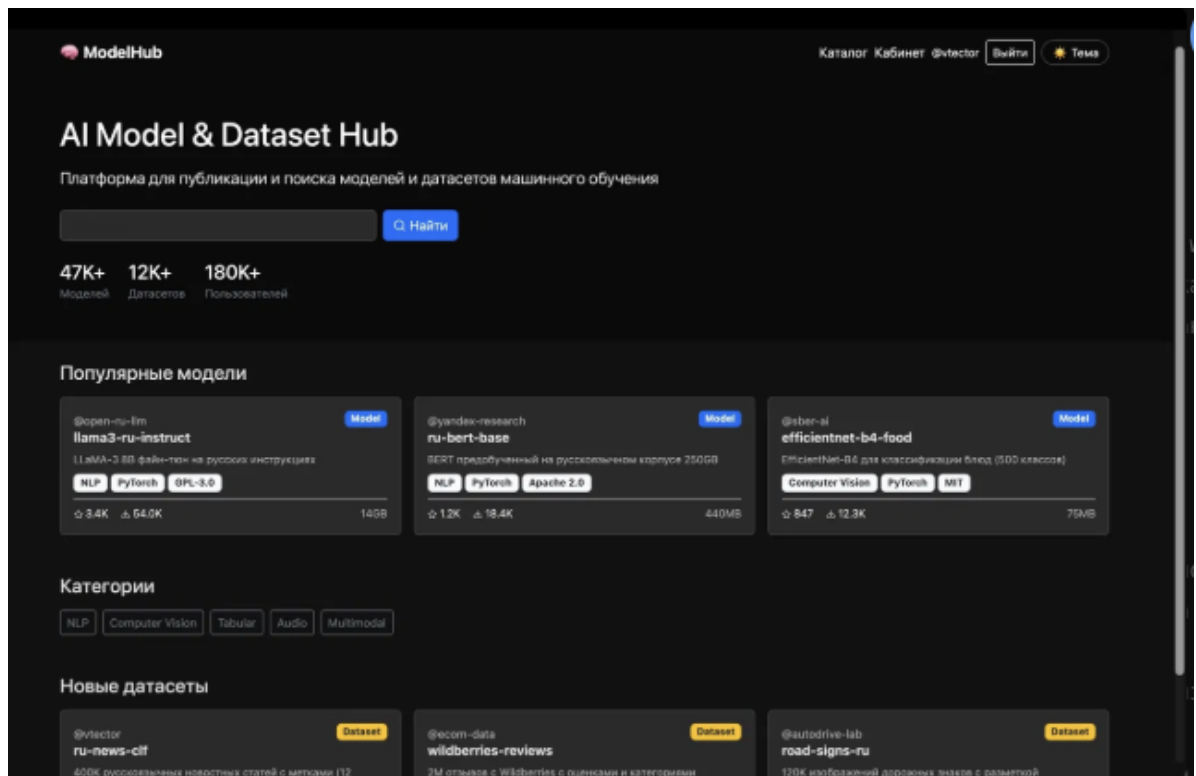


Рис. 2: Главная страница с загруженными данными из API. Карточки моделей получены через GET /models к json-server.

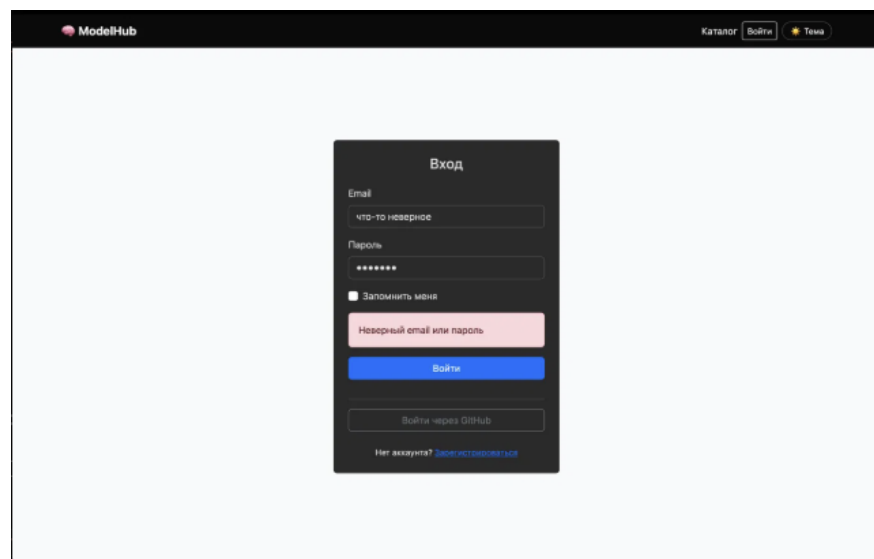


Рис. 3: Сервер вернул ошибку 401 при неверных данных входа. В ЛР1 такой проверки не было — любые данные принимались.

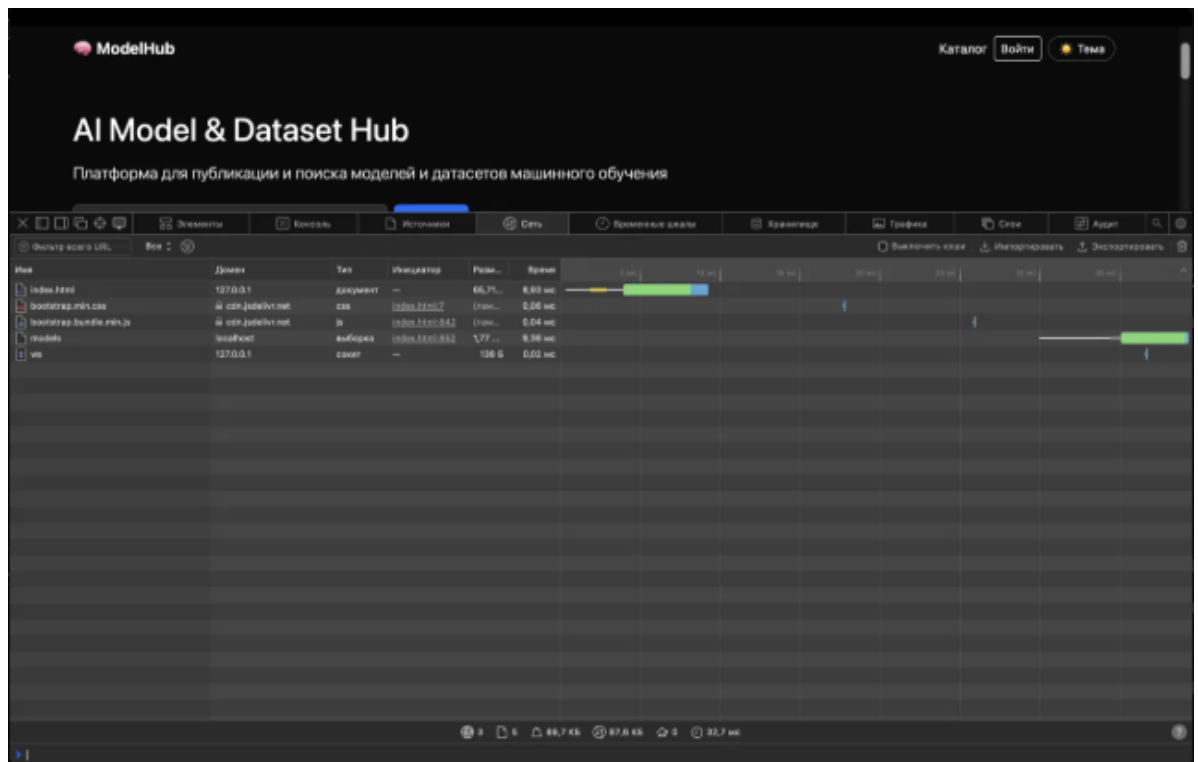


Рис. 4: DevTools → Network: виден запрос models к localhost (json-server на порту 3001). Время ответа 9.36 мс. Тип — выборка (fetch).

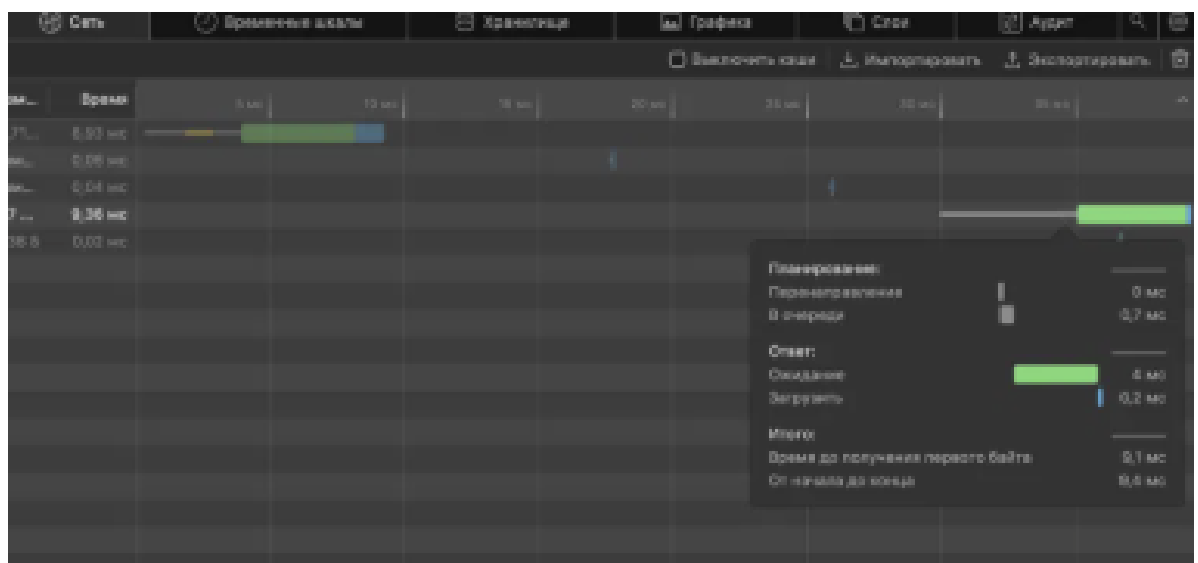


Рис. 5: Детали запроса models: домен localhost, время ожидания 4 мс, загрузка 0.2 мс.

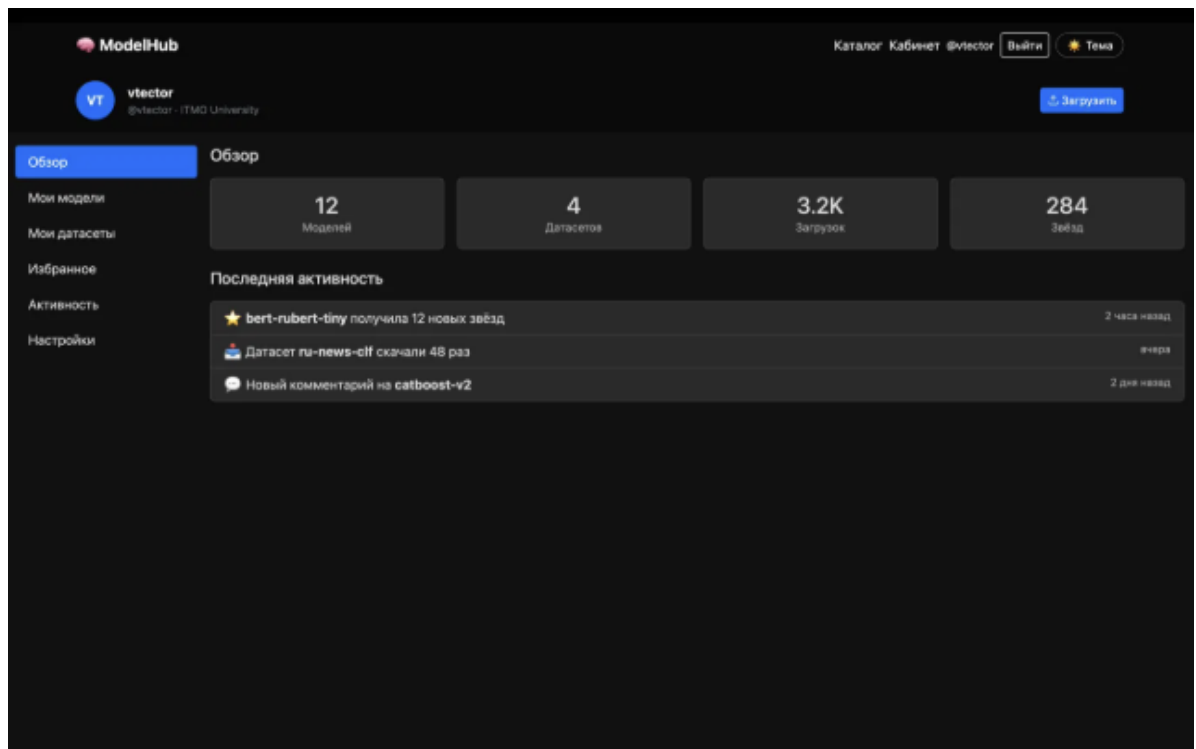


Рис. 6: Личный кабинет после успешного входа. Токен сохранён в `localStorage` и восстанавливается при перезагрузке страницы.

Вывод

В ходе лабораторной работы приложение AI Model & Dataset Hub подключено к моковому API на основе `json-server`.

Что реализовано:

- Сервер на `localhost:3001` с базой данных в `db.json`
- `GET /models` — загрузка всех моделей и датасетов
- `POST /login` — авторизация с проверкой email/пароля, возврат токена
- `POST /register` — регистрация нового пользователя
- `GET /me` — проверка текущей сессии по токену
- `POST /logout` — удаление токена
- Функция `apiFetch` автоматически добавляет токен ко всем запросам
- Сессия восстанавливается после перезагрузки через `localStorage`

Ограничения: пароли хранятся в открытом виде в `db.json` — в реальном приложении необходимо хеширование. Функционал загрузки и скачивания файлов не реализован. В ЛРЗ приложение мигрирует на `Vue.js` с `axios` вместо `fetch`.